

SE446: Big Data Engineering

Week 6A: YARN — Cluster Resource Management

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture
- 4 YARN Application Lifecycle
- 5 Failure Handling
- 6 YARN Schedulers
- 7 YARN CLI & Web UI
- 8 YARN in Context
- 9 Summary

Outline

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture
- 4 YARN Application Lifecycle
- 5 Failure Handling
- 6 YARN Schedulers
- 7 YARN CLI & Web UI
- 8 YARN in Context
- 9 Summary

Recap: What We Have So Far

We learned to:

- ✓ Store data in **HDFS** (Week 2)
- ✓ Process data with **MapReduce** (Weeks 3–4)
- ✓ Query data with **Hive SQL** (Week 5)

But we skipped something:

- ? Who allocates CPUs and RAM?
- ? Who launches our tasks on workers?
- ? Who restarts failed containers?

The Missing Piece

Every MapReduce job and every Hive query you ran this semester was managed by a system called **YARN**. Today we finally meet it.

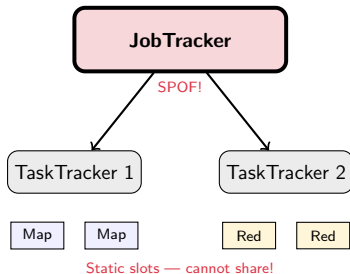
The Problem: Hadoop 1.0 Had a Single Boss

Hadoop 1.0 architecture (MRv1):

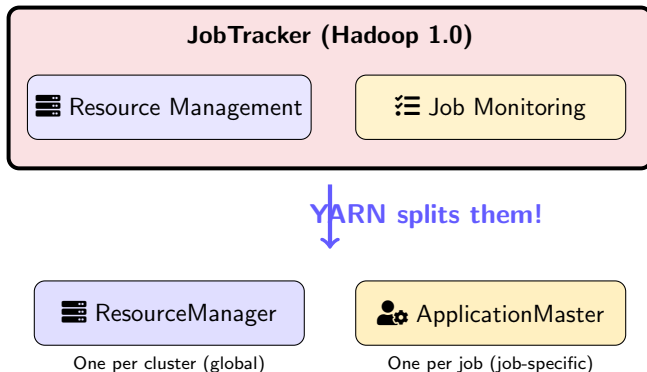
- One **JobTracker** managed *everything*:
 - Cluster resource allocation
 - Task scheduling for all jobs
 - Fault tolerance and retries
- Multiple **TaskTrackers** reported to it

Problems:

- 1 **Single point of failure** — crash = cluster down
- 2 **Max ~4,000 nodes** — scalability ceiling
- 3 **MapReduce only** — no Spark, no Hive-on-Tez
- 4 **Static slots** — wasted resources when idle



The Two Responsibilities That Needed Separation



Key Insight

YARN decouples **cluster resource management** from **per-job task tracking**. Any framework (MapReduce, Spark, Hive, Flink) can plug into YARN for resources.

Outline

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture
- 4 YARN Application Lifecycle
- 5 Failure Handling
- 6 YARN Schedulers
- 7 YARN CLI & Web UI
- 8 YARN in Context
- 9 Summary

YARN: Yet Another Resource Negotiator

Definition

YARN is the **cluster resource manager** in Hadoop 2.0+. It allocates CPU, memory, and disk to applications, and monitors their execution.

What YARN does:

- ✓ Manages cluster resources (CPU + RAM)
- ✓ Launches and monitors application containers
- ✓ Supports multiple frameworks simultaneously
- ✓ Handles failures and restarts

What YARN does NOT do:

- ✗ Store data (that's HDFS)
- ✗ Parse queries (that's Hive Driver)
- ✗ Know about MapReduce internals
- ✗ Understand Spark RDDs

Analogy

YARN is like an **airport control tower**: it assigns runways (resources) to planes (jobs), but it doesn't fly the planes or know what cargo they carry.

Hadoop 1.0 vs. YARN: Side-by-Side

Aspect	Hadoop 1.0 (MRv1)	YARN (Hadoop 2.0+)
Resource Manager	JobTracker (single)	ResourceManager (HA)
Resource Units	Static Map/Reduce slots	Dynamic containers
Job Tracking	JobTracker	Per-job ApplicationMaster
Frameworks	MapReduce only	MR, Spark, Hive, Flink, ...
Max Cluster Size	~4,000 nodes	10,000+ nodes
SPOF	 Yes	 No (HA)
Resource Flexibility	Pre-configured	On-demand allocation

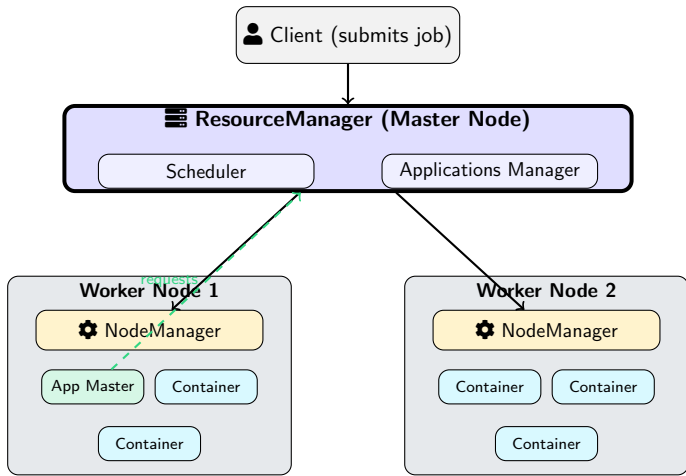
Critical Improvement

YARN turns Hadoop from a **MapReduce-only system** into a **general-purpose cluster OS** that can run any distributed framework.

Outline

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture**
- 4 YARN Application Lifecycle
- 5 Failure Handling
- 6 YARN Schedulers
- 7 YARN CLI & Web UI
- 8 YARN in Context
- 9 Summary

YARN Architecture: The Big Picture



Component 1: ResourceManager — The Cluster Brain

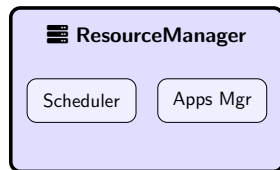
The ResourceManager runs on the **master node** and has two sub-components:

1. Scheduler

- Allocates resources (containers) to applications
- Does **NOT** monitor tasks or handle retries
- Pluggable: FIFO, Fair, or Capacity

2. Applications Manager (AsM)

- Accepts job submissions from clients
- Allocates the **first container** for the ApplicationMaster
- Restarts the AM if it crashes



Web UI: <http://master:8088>

Key Point

The ResourceManager **does not know** what your job is doing. It only manages resources — the ApplicationMaster handles job logic.

Component 2: NodeManager — Worker Node Agent

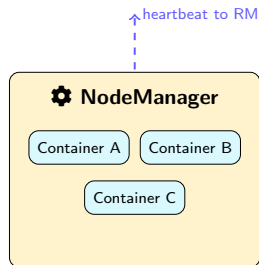
A NodeManager runs on **every worker node** in the cluster (we have 2).

Responsibilities:

- 1 **Heartbeat**: Reports available CPU + RAM to ResourceManager
- 2 **Launch**: Creates containers on request from RM
- 3 **Monitor**: Tracks container resource usage
- 4 **Kill**: Terminates containers exceeding limits
- 5 **Report**: Status updates to both RM and AM

Important

NodeManagers do **NOT** communicate directly with each other. All coordination goes through the ResourceManager.



Component 3: ApplicationMaster — The Job's Own Manager

One AM Per Running Application

When you submit a job, YARN launches an **ApplicationMaster** inside a container on one of the worker nodes. This AM manages **only that one job**.

AM responsibilities:

- 1 Negotiate additional containers from the ResourceManager
- 2 Launch tasks inside those containers (via NodeManagers)
- 3 Monitor task progress and handle retries
- 4 Report final status (SUCCESS / FAILED) to the ResourceManager

AM is framework-specific:

Framework	AM Class	What It Manages
MapReduce	MRAppMaster	Map and Reduce tasks
Hive (Tez)	DAGAppMaster	Tez DAG operators
Spark	SparkAM	Spark executors

Key Insight

YARN doesn't need to understand MapReduce, Spark, or Hive — it just gives them containers. The AM handles everything else.

Component 4: Containers — The Unit of Work

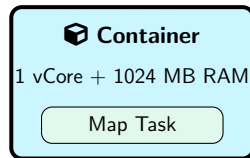
A **container** is a bundle of resources on one NodeManager:

- **vCores**: virtual CPU cores (e.g., 1)
- **Memory**: RAM in MB (e.g., 1024)

One task runs per container:

- Map task container: 1 vCore, 1 GB
- Reduce task container: 1 vCore, 1 GB
- ApplicationMaster container: 1 vCore, 512 MB
- Spark executor: 2 vCores, 4 GB

Containers are **non-reusable**: task completes → container released.



Resource Limits

If a container exceeds its allocated memory, the NodeManager **kills it** immediately.

Our Cluster's Resource Configuration

SE446 Lab Cluster: 1 Master + 2 Workers

Configuration	Per Worker	Cluster Total
<code>resource.memory-mb</code>	4096 MB	8192 MB (8 GB)
<code>resource.cpu-vcores</code>	4	8
<code>minimum-allocation-mb</code>		512 MB
<code>minimum-allocation-vcores</code>		1

Capacity Calculation

If each container needs 512 MB and 1 vCore:

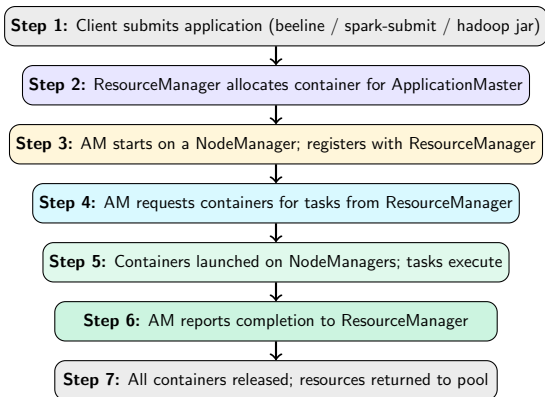
Max containers = $\min\left(\frac{8192}{512}, \frac{8}{1}\right) = \min(16, 8) = 8$ containers simultaneously (memory allows 16, but only 8 vCores).

Configuration file: `/etc/hadoop/conf/yarn-site.xml`

Outline

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture
- 4 YARN Application Lifecycle**
- 5 Failure Handling
- 6 YARN Schedulers
- 7 YARN CLI & Web UI
- 8 YARN in Context
- 9 Summary

YARN Application Lifecycle: 7 Steps



Example: Tracing a Hive Query Through YARN

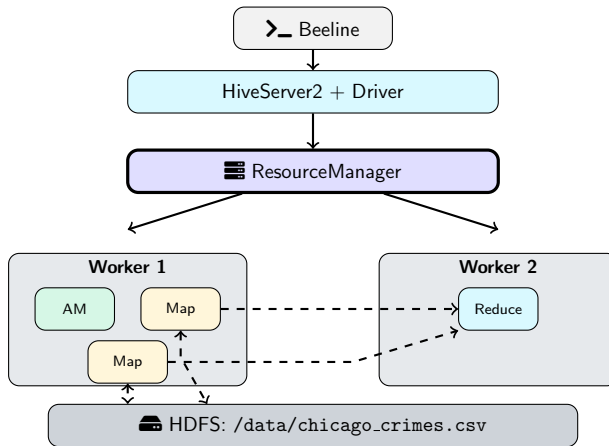
Query: `SELECT primary_type, COUNT(*) FROM crimes GROUP BY primary_type;`

- 1  **Client:** Beeline sends query to HiveServer2
- 2  **Hive Driver:** Parses SQL → MapReduce/Tez plan
- 3  **YARN RM:** Assigns 1 container for AM on Worker 1
- 4  **AM:** Requests 2 Map containers + 1 Reduce container
- 5  **NMs:** Launch the 3 task containers on workers
- 6  **Tasks run:**
 - Map tasks read HDFS blocks, emit (`crime_type`, 1)
 - Shuffle groups by key
 - Reduce task sums counts per crime type
- 7  **AM:** Reports SUCCESS to ResourceManager
- 8  **Beeline:** Displays results to user

Lab Connection

Open `http://134.209.172.50:8088` while querying — you'll see this exact lifecycle unfold in the YARN Web UI.

Visual: A Hive Job on Our Cluster



Outline

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture
- 4 YARN Application Lifecycle
- 5 Failure Handling**
- 6 YARN Schedulers
- 7 YARN CLI & Web UI
- 8 YARN in Context
- 9 Summary

What Happens When Things Fail?

1. Container Failure

- AM detects via heartbeat timeout
- AM requests a **replacement container** from RM
- Task re-executed on new container
- ✓ Automatic recovery

2. ApplicationMaster Failure

- RM detects AM is unresponsive
- RM restarts AM (up to 2 attempts by default)
- Config: `am.max-attempts`
- ✓ Automatic recovery

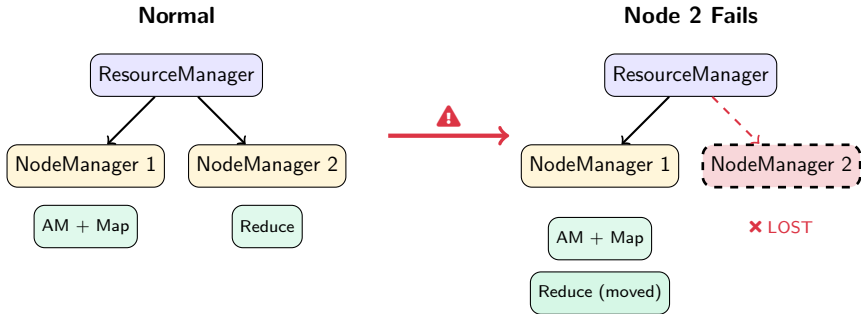
3. NodeManager Failure

- RM detects missed heartbeats
- Marks node as “LOST”
- All containers on that node re-scheduled to other nodes
- ✓ Automatic recovery

4. ResourceManager Failure

- ⚠ Cluster stops accepting new jobs
- Solution: RM High Availability
- Active + Standby RM with ZooKeeper
- Our lab: single RM (no HA)

Failure Recovery — Visual



Takeaway

YARN automatically reschedules containers from a failed node to a healthy one. Data is safe because it's replicated across HDFS DataNodes.

Outline

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture
- 4 YARN Application Lifecycle
- 5 Failure Handling
- 6 YARN Schedulers**
- 7 YARN CLI & Web UI
- 8 YARN in Context
- 9 Summary

The Scheduling Problem

Scenario

3 student teams submit Hive queries at the same time on our 2-worker cluster.

Who gets resources first? How much? For how long?

YARN provides three built-in scheduler strategies:

- ① **FIFO Scheduler** — First come, first served
- ② **Fair Scheduler** — Equal shares for all running jobs
- ③ **Capacity Scheduler** — Queue-based guaranteed capacity

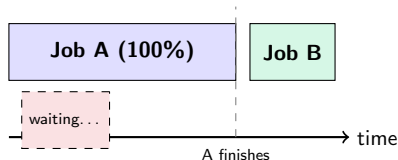
FIFO Scheduler — Simple but Unfair

Strategy: First-submitted job gets **all** cluster resources. Next job waits until the first finishes.

Problem:

- One large job **blocks** all others
- A 30-minute MR job prevents a 10-second query from running
- Head-of-line blocking

Use case: Testing / development only

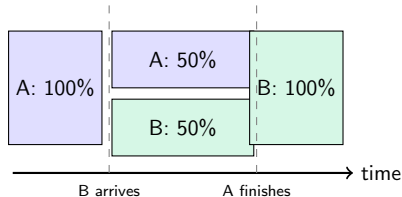


Fair Scheduler — Equal Shares

Strategy: All running jobs get **equal** resource shares.

How it works:

- 1 job running → gets 100%
- 2 jobs arrivals → each gets 50%
- 3 jobs → each gets 33%
- When a job finishes, others expand



Use case: Multi-user shared clusters (like ours!)

Advantage

Short jobs don't get stuck behind long jobs — they share the cluster instantly.

Capacity Scheduler — Queue-Based (Default)

Strategy: Cluster divided into **queues** with guaranteed capacity.

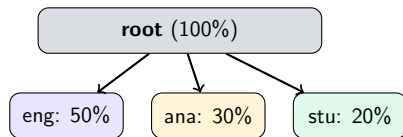
Example queue tree:

- root/
 - engineering — 50% capacity
 - analytics — 30% capacity
 - students — 20% capacity

Key features:

- Each queue guaranteed its configured %
- Queues can *borrow* unused capacity
- Within a queue: Fair Scheduler

Our cluster: single default queue at 100%



Scheduler Comparison — Which to Use?

Feature	FIFO	Fair	Capacity
Strategy	First-come	Equal share	Queue-based
Multi-job support	✗	✓	✓
Resource isolation	None	Soft	Hard (configurable)
Priority support	✗	Weighted	Queue priority
Best for	Dev/test	Shared clusters	Enterprise
Default in YARN	✗	✗	✓

Our Lab Cluster

Uses **Capacity Scheduler** with one default queue. When all students submit jobs at once, they share this single queue's resources fairly.

Outline

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture
- 4 YARN Application Lifecycle
- 5 Failure Handling
- 6 YARN Schedulers
- 7 YARN CLI & Web UI**
- 8 YARN in Context
- 9 Summary

YARN CLI: Essential Commands

```
# List all running applications
yarn application -list

# List ALL applications (including finished)
yarn application -list -appStates ALL

# Get detailed status of one application
yarn application -status application_1234567890_0001

# Kill a runaway job
yarn application -kill application_1234567890_0001

# Retrieve application logs (after completion)
yarn logs -applicationId application_1234567890_0001
```

```
# Check cluster node resources
yarn node -list -all

# Check queue status
yarn queue -status default
```

Live Example: Monitoring a Hive Job

Step 1: Submit a Hive query in one terminal:

```
beeline -u 'jdbc:hive2://' -e "SELECT primary_type, COUNT(*)  
FROM crimes GROUP BY primary_type ORDER BY COUNT(*) DESC;"
```

Step 2: In another terminal, watch YARN:

```
# While query is running:  
yarn application -list  
# Output:  
# application_17092..._0042    Hive    RUNNING    default  
#   AM Host: worker1    Containers: 4    Memory: 4096 MB
```

Step 3: After completion, get logs:

```
yarn logs -applicationId application_17092..._0042 | head -30
```

Lab Exercise

You will do exactly this in today's lab. Record the Application ID, AM host, container count, and memory used.

YARN Web UI: Key Pages

URL: `http://134.209.172.50:8088`

Tab	What to Look For
All Applications	App ID, name, type, state, running containers, allocated memory/vCores, elapsed time
Nodes	Per-node: total and available memory, vCores, number of running containers
Scheduler	Queue tree: used vs. available capacity, number of running applications per queue
App Detail	AM host, number of attempts, list of containers, link to job-specific UI (MapReduce / Spark)
App Logs	Per-container stdout/stderr output — useful for debugging failures



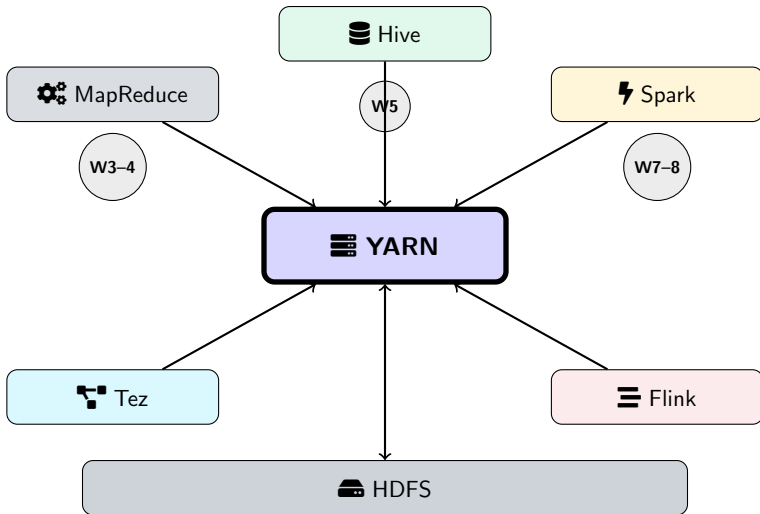
Tip

The “Tracking URL” link inside the Application detail page takes you to the **framework-specific UI** (e.g., Spark Web UI at port 4040 or MapReduce Job History).

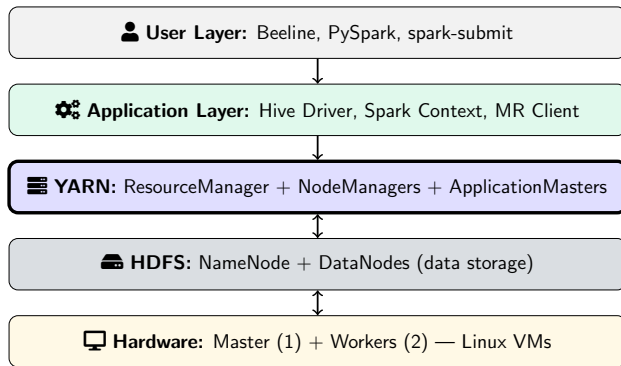
Outline

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture
- 4 YARN Application Lifecycle
- 5 Failure Handling
- 6 YARN Schedulers
- 7 YARN CLI & Web UI
- 8 YARN in Context**
- 9 Summary

YARN Powers Everything We Run



The Full Stack: How It All Connects



Understanding YARN = Understanding Your Cluster

When a query is slow, check YARN first: Are containers waiting? Is memory full? Is another job hogging the queue?

Outline

- 1 Recap & Motivation
- 2 What Is YARN?
- 3 YARN Architecture
- 4 YARN Application Lifecycle
- 5 Failure Handling
- 6 YARN Schedulers
- 7 YARN CLI & Web UI
- 8 YARN in Context
- 9 Summary

Key Takeaways

- 1 **YARN** = Cluster resource manager in Hadoop 2.0+
- 2 **ResourceManager**: Global scheduler + applications manager (master node)
- 3 **NodeManager**: Worker agent that launches/monitors containers
- 4 **ApplicationMaster**: Per-job manager (framework-specific)
- 5 **Container**: Unit of allocated resources (vCores + RAM)
- 6 **Lifecycle**: Client → RM → AM → Containers → Done
- 7 **Schedulers**: FIFO (dev), Fair (shared), Capacity (enterprise)
- 8 **Monitoring**: Web UI at port 8088 + yarn CLI

Remember

HDFS	Where the data lives
YARN	Who runs the computations
Hive/MR/Spark	What the computation is

Next Session: Advanced HiveQL

Week 6B: Advanced HiveQL — NYC Taxi Analytics

- New dataset: **NYC Yellow Taxi** trips (10,000 rows)
- Advanced GROUP BY: ROLLUP and CUBE subtotals
- Window functions: RANK, LAG, LEAD, NTILE
- Multi-table JOIN with lookup tables
- EXPLAIN plans + ORC vs TextFile performance

Pre-Class Preparation

- SSH into the cluster: `ssh root@134.209.172.50`
- Open YARN Web UI: `http://134.209.172.50:8088`
- Review SQL window functions (if unfamiliar)

Pre-Class Preparation for Lab

Today's lab: YARN Monitoring

- Open the YARN Web UI at `http://134.209.172.50:8088`
- Submit a Hive query via Beeline
- While it runs, observe the application appear in YARN
- Record: Application ID, AM host, containers, memory
- After completion: retrieve logs with `yarn logs`

Reminder

Midterm 1 covers Weeks 1–6 (HDFS, MapReduce, Hive, YARN).
Understanding YARN is critical for the exam.